

Module 8: Cluster Analysis

The following tutorial contains Python examples for solving clustering problems. You should refer to Chapters 7 and 8 of the "Introduction to Data Mining" book to understand some of the concepts introduced in this tutorial. The notebook can be downloaded from <http://www.cse.msu.edu/~ptan/dmbook/tutorials/tutorial8/tutorial8.ipynb>.

Cluster analysis seeks to partition the input data into groups of closely related instances so that instances that belong to the same cluster are more similar to each other than to instances that belong to other clusters. In this tutorial, we will provide examples of using different clustering techniques provided by the scikit-learn library package.

Read the step-by-step instructions below carefully. To execute the code, click on the corresponding cell and press the SHIFT-ENTER keys simultaneously.

```
import numpy as np
import pandas as pd
import math
from sklearn import cluster
import matplotlib.pyplot as plt
%matplotlib inline
from scipy.cluster import hierarchy
from sklearn.cluster import DBSCAN, k_means, KMeans
from sklearn.preprocessing import StandardScaler, normalize
```

8.1 K-means Clustering

The k-means clustering algorithm represents each cluster by its corresponding cluster centroid. The algorithm would partition the input data into k disjoint clusters by iteratively applying the following two steps:

1. Form k clusters by assigning each instance to its nearest centroid.
2. Recompute the centroid of each cluster.

In this section, we perform k-means clustering on a toy example of movie ratings dataset. We first create the dataset as follows.

```
ratings = [['john',5,5,2,1],['mary',4,5,3,2],['bob',4,4,4,3],['lisa',2,2,4,5],['lee',1,2,3,4],['harry',2,1,5,5]]
titles = ['user','Jaws','Star Wars','Exorcist','Omen']
movies = pd.DataFrame(ratings,columns=titles)
movies
```

	user	Jaws	Star Wars	Exorcist	Omen
0	john	5	5	2	1
1	mary	4	5	3	2
2	bob	4	4	4	3
3	lisa	2	2	4	5
4	lee	1	2	3	4
5	harry	2	1	5	5

In this example dataset, the first 3 users liked action movies (Jaws and Star Wars) while the last 3 users enjoyed horror movies (Exorcist and Omen). Our goal is to apply k-means clustering on the users to identify groups of users with similar movie preferences.

The example below shows how to apply k-means clustering (with $k=2$) on the movie ratings data. We must remove the "user" column first before applying the clustering algorithm. The cluster assignment for each user is displayed as a dataframe object.

```
data = movies.drop('user',axis=1)
k_means = cluster.KMeans(n_clusters=2, max_iter=50, random_state=1)
k_means.fit(data)
labels = k_means.labels_
pd.DataFrame(labels, index=movies.user, columns=['Cluster ID'])
```

Cluster ID

user

john	0
mary	0
bob	0
lisa	1
lee	1
harry	1

The k-means clustering algorithm assigns the first three users to one cluster and the last three users to the second cluster. The results are consistent with our expectation. We can also display the centroid for each of the two clusters.

```
centroids = k_means.cluster_centers_  
pd.DataFrame(centroids,columns=data.columns)
```

	Jaws	Star Wars	Exorcist	Omen
0	4.333333	4.666667	3.0	2.000000
1	1.666667	1.666667	4.0	4.666667

Observe that cluster 0 has higher ratings for the horror movies whereas cluster 1 has higher ratings for action movies. The cluster centroids can be applied to other users to determine their cluster assignments.

```
testData = np.array([[4,5,1,2],[3,2,4,4],[2,3,4,1],[3,2,3,3],[5,4,1,4]])  
labels = k_means.predict(testData)  
labels = labels.reshape(-1,1)  
usernames = np.array(['paul','kim','liz','tom','bill']).reshape(-1,1)  
cols = movies.columns.tolist()  
cols.append('Cluster ID')  
newusers = pd.DataFrame(np.concatenate((usernames, testData, labels), axis=1),columns=cols)  
newusers
```

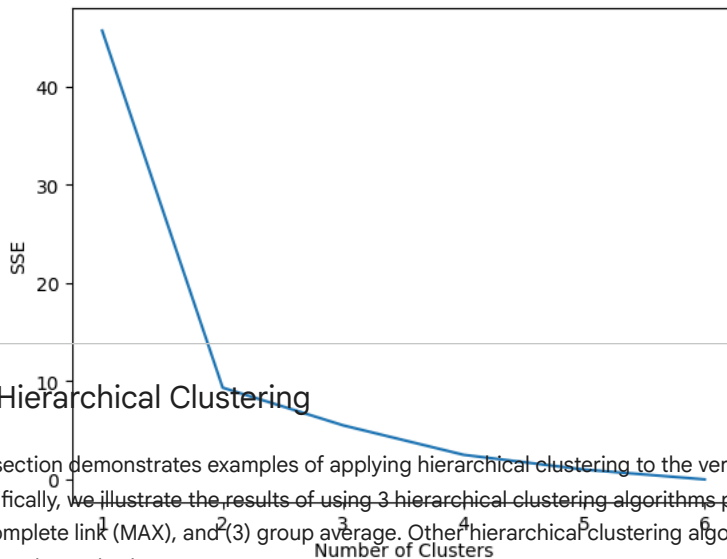
```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but KMeans was fitted  
warnings.warn(
```

	user	Jaws	Star Wars	Exorcist	Omen	Cluster	ID
0	paul	4		5	1	2	0
1	kim	3		2	4	4	1
2	liz	2		3	4	1	0
3	tom	3		2	3	3	1
4	bill	5		4	1	4	0

To determine the number of clusters in the data, we can apply k-means with varying number of clusters from 1 to 6 and compute their corresponding sum-of-squared errors (SSE) as shown in the example below. The "elbow" in the plot of SSE versus number of clusters can be used to estimate the number of clusters.

```
numClusters = [1,2,3,4,5,6]  
SSE = []  
for k in numClusters:  
    k_means = cluster.KMeans(n_clusters=k)  
    k_means.fit(data)  
    SSE.append(k_means.inertia_)  
  
plt.plot(numClusters, SSE)  
plt.xlabel('Number of Clusters')  
plt.ylabel('SSE')
```

Text(0, 0.5, 'SSE')



8.2 Hierarchical Clustering

This section demonstrates examples of applying hierarchical clustering to the vertebrate dataset used in Module 6 (Classification). Specifically, we illustrate the results of using 3 hierarchical clustering algorithms provided by the Python scipy library: (1) single link (MIN), (2) complete link (MAX), and (3) group average. Other hierarchical clustering algorithms provided by the library include centroid-based and Ward's method.

```
# import kagglehub

# # Download latest version
# path = kagglehub.dataset_download("almutairiphd/vertebrate")

# print("Path to dataset files:", path)

data = pd.read_csv('/content/vertebrate.csv')
data.head()
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipython-input-2793375037.py in <cell line: 0>()
      6 # print("Path to dataset files:", path)
      7
----> 8 data = pd.read_csv('/content/vertebrate.csv')
      9 data.head()
```

```
----- 4 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/io/common.py in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text,
errors, storage_options)
    871     if ioargs.encoding and "b" not in ioargs.mode:
    872         # Encoding
--> 873         handle = open(
    874             handle,
    875             ioargs.mode,
```

```
FileNotFoundError: [Errno 2] No such file or directory: '/content/vertebrate.csv'
```

```
names = data['Name']
Y = data['Class']
X = data.drop(['Name', 'Class'], axis=1)
Z = hierarchy.linkage(X, 'single')
#dn = hierarchy.dendrogram(Z, labels=names.tolist(), orientation='right')

plt.figure(figsize=(10, 5)) # Adjust the width and height
dn = hierarchy.dendrogram(Z, labels=names.tolist(), leaf_font_size=8)
plt.xticks(rotation=90)
plt.show()
```

8.2.2 Complete Link (MAX)

```
Z = hierarchy.linkage(X, 'complete')
dn = hierarchy.dendrogram(Z, labels=names.tolist(), orientation='right')
```

8.3.3 Group Average

```
Z = hierarchy.linkage(X, 'average')
dn = hierarchy.dendrogram(Z, labels=names.tolist(), orientation='right')
```

8.3 Density-Based Clustering

Density-based clustering identifies the individual clusters as high-density regions that are separated by regions of low density. DBSCAN is one of the most popular density based clustering algorithms. In DBSCAN, data points are classified into 3 types---core points, border points, and noise points---based on the density of their local neighborhood. The local neighborhood density is defined according to 2 parameters: radius of neighborhood size (eps) and minimum number of points in the neighborhood (min_samples).

For this approach, we will use a noisy, 2-dimensional dataset originally created by Karypis et al. [1] for evaluating their proposed CHAMELEON algorithm. The example code shown below will load and plot the distribution of the data.

```
data = pd.read_csv('http://pluto.hood.edu/~dong/datasets/chameleon.data', delimiter=' ', names=['x','y'])
data.plot.scatter(x='x',y='y')
```

We apply the DBSCAN clustering algorithm on the data by setting the neighborhood radius (eps) to 15.5 and minimum number of points (min_samples) to be 5. The clusters are assigned to IDs between 0 to 8 while the noise points are assigned to a cluster ID equals to -1.

```
result['Cluster ID'].value_counts()
```

```
print(data.shape)
db = DBSCAN(eps=15.5, min_samples=5).fit(data)
#core_samples_mask = np.zeros_like(db.labels_, dtype=bool)
#core_samples_mask[db.core_sample_indices_] = True
#print(len(core_samples_mask[core_samples_mask == True]))

labels = pd.DataFrame(db.labels_,columns=['Cluster ID'])
result = pd.concat((data,labels), axis=1)
result.plot.scatter(x='x',y='y',c='Cluster ID', colormap='jet')
```

8.4 Spectral Clustering

One of the main limitations of the k-means clustering algorithm is its tendency to seek for globular-shaped clusters. Thus, it does not work when applied to datasets with arbitrary-shaped clusters or when the cluster centroids overlapped with one another. Spectral clustering can overcome this limitation by exploiting properties of the similarity graph to overcome such limitations. To illustrate this, consider the following two-dimensional datasets.

```
import pandas as pd

data1 = pd.read_csv('http://pluto.hood.edu/~dong/datasets/2d_data.txt', delimiter=' ', names=['x','y'])
data2 = pd.read_csv('http://pluto.hood.edu/~dong/datasets/elliptical.txt', delimiter=' ', names=['x','y'])

fig, (ax1,ax2) = plt.subplots(nrows=1, ncols=2, figsize=(12,5))
data1.plot.scatter(x='x',y='y',ax=ax1)
data2.plot.scatter(x='x',y='y',ax=ax2)
```

Below, we demonstrate the results of applying k-means to the datasets (with k=2).

```
from sklearn import cluster

k_means = cluster.KMeans(n_clusters=2, max_iter=50, random_state=1, n_init = 'auto')
k_means.fit(data1)
labels1 = pd.DataFrame(k_means.labels_,columns=['Cluster ID'])
result1 = pd.concat((data1,labels1), axis=1)

k_means2 = cluster.KMeans(n_clusters=2, max_iter=50, random_state=1, n_init = 'auto')
k_means2.fit(data2)
labels2 = pd.DataFrame(k_means2.labels_,columns=['Cluster ID'])
result2 = pd.concat((data2,labels2), axis=1)

fig, (ax1,ax2) = plt.subplots(nrows=1, ncols=2, figsize=(12,5))
result1.plot.scatter(x='x',y='y',c='Cluster ID',colormap='jet',ax=ax1)
ax1.set_title('K-means Clustering')
result2.plot.scatter(x='x',y='y',c='Cluster ID',colormap='jet',ax=ax2)
ax2.set_title('K-means Clustering')
```

The plots above show the poor performance of k-means clustering. Next, we apply spectral clustering to the datasets. Spectral clustering converts the data into a similarity graph and applies the normalized cut graph partitioning algorithm to generate the clusters. In the example below, we use the Gaussian radial basis function as our affinity (similarity) measure. Users need to tune the kernel parameter (gamma) value in order to obtain the appropriate clusters for the given dataset.

```
from sklearn import cluster
import pandas as pd
```

```
spectral = cluster.SpectralClustering(n_clusters=2,random_state=1,affinity='rbf',gamma=5000)
spectral.fit(data1)
labels1 = pd.DataFrame(spectral.labels_,columns=['Cluster ID'])
result1 = pd.concat((data1,labels1), axis=1)

spectral2 = cluster.SpectralClustering(n_clusters=2,random_state=1,affinity='rbf',gamma=1000)
spectral2.fit(data2)
labels2 = pd.DataFrame(spectral2.labels_,columns=['Cluster ID'])
result2 = pd.concat((data2,labels2), axis=1)

fig, (ax1,ax2) = plt.subplots(nrows=1, ncols=2, figsize=(12,5))
result1.plot.scatter(x='x',y='y',c='Cluster ID',colormap='jet',ax=ax1)
ax1.set_title('Spectral Clustering')
result2.plot.scatter(x='x',y='y',c='Cluster ID',colormap='jet',ax=ax2)
ax2.set_title('Spectral Clustering')
```

8.5 Summary

This tutorial illustrates examples of using different Python's implementation of clustering algorithms. Algorithms such as k-means, spectral clustering, and DBScan are designed to create disjoint partitions of the data whereas the single-link, complete-link, and group average algorithms are designed to generate a hierarchy of cluster partitions.

References: [1] George Karypis, Eui-Hong Han, and Vipin Kumar. CHAMELEON: A Hierarchical Clustering Algorithm Using Dynamic Modeling. IEEE Computer 32(8): 68-75, 1999.

Homework 5

1. Given the file college-and-university.csv (see the dataset folder: <http://pluto.hood.edu/~dong/datasets/>), conduct K-means clustering analysis based on Median SAT, Acceptance Rate, Expenditures/Student, Top 10% HS, and Graduation %. Do not use Type and School in your analysis. Determine the optimal value of K.
2. Using college-and-university.csv, conduct Hierarchical clustering analysis based on Median SAT, Acceptance Rate, Expenditures/Student, Top 10% HS, and Graduation %. Do not use Type and School in your analysis. **Show at least two hierarchical algorithms. Compare and contrast the results.**
3. Using the data in df below, conduct K-means and density-based clustering. Provide visualizations of the clustering results. Use eps=30 and min_samples=6 for DBScan. **You are encouraged to experiment with eps and min_samples to optimize the results.**

```
# imports for models
import pandas as pd
import numpy as np
import math
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.cluster import DBSCAN, KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.cluster import SpectralClustering
from sklearn.decomposition import PCA
```

```
data = pd.read_csv('/content/colleges-and-universities.csv')
data
```

Next steps:

[Generate code with data](#)

[New interactive sheet](#)

▼




```
# problem 1 solution

# conduct K-means clustering analysis based on Median SAT, Acceptance Rate, Expenditures/Student, Top 10% HS, and Graduation %. Do not use Typ
X = data[['Median SAT', 'Acceptance Rate', 'Expenditures/Student', 'Top 10% HS', 'Graduation %']].copy()
X['Acceptance Rate'] = X['Acceptance Rate'].str.replace('%', '').astype(float)

n_clusters = 3 # this is optimal val from below

# scale data & fit to model
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
kmeans = KMeans(n_clusters=n_clusters, random_state=0, n_init='auto')
kmeans.fit(X)

# cluster assignments
labels = kmeans.labels_
result = data.copy()
result['Cluster ID'] = labels
display(result[['School', 'Cluster ID']])
```

14	Cornell	University	1280	30%	37137.0	85	83
15	Davidson	Lib Arts	1230	36%	17721.0	77	89
16	Duke	University	1310	25%	39504.0	91	91
17	Georgetown	University	1278	24%	23115.0	79	89
18	Grinnell	Lib Arts	1244	67%	22301.0	65	73
19	Hamilton	Lib Arts	1215	38%	20722.0	51	85
20	Harvard	University	1370	18%	46918.0	90	90
21	Haverford	Lib Arts	1285	35%	19418.0	71	87
22	Johns Hopkins	University	1290	48%	45460.0	69	86
23	Middlebury	Lib Arts	1255	25%	24718.0	65	92
24	MIT	University	1357	30%	56766.0	95	86
25	Mount Holyoke	Lib Arts	1200	61%	23358.0	47	83
26	Northwestern	University	1230	47%	28851.0	77	82
27	Oberlin	Lib Arts	1247	54%	23591.0	64	77
28	Occidental	Lib Arts	1170	49%	20192.0	54	72
29	Pomona	Lib Arts	1320	33%	26668.0	79	80
30	Princeton	University	1340	17%	48123.0	89	93
31	Rice	University	1327	24%	26730.0	85	88
32	Smith	Lib Arts	1195	57%	25271.0	65	87
33	Stanford	University	1370	18%	61921.0	92	88
34	Swarthmore	Lib Arts	1310	24%	27487.0	78	88
35	U Michigan	University	1195	60%	21853.0	71	77
36	U of Chicago	University	1300	45%	38937.0	74	73
37	U of Rochester	University	1155	56%	38597.0	52	73
38	U Pennsylvania	University	1280	41%	30882.0	87	86
39	U Va	University	1218	37%	19365.0	77	88
40	UCLA	University	1142	43%	26859.0	96	61
41	UNC	University	1109	32%	19684.0	82	73
42	Vassar	Lib Arts	1287	43%	20179.0	53	84
43	Washington U (MO)	University	1225	54%	39883.0	71	76
44	Washinton and Lee	Lib Arts	1234	29%	17998.0	61	78
45	Wellesley	Lib Arts	1250	49%	27879.0	76	86
46	Wesleyan (CT)	Lib Arts	1290	35%	19948.0	73	91
47	Williams	Lib Arts	1336	28%	23772.0	86	93
48	Yale	University	1350	19%	52468.0	90	93


```

inertia = []
silhouette = []

K_range = range(2, 10)
for k in K_range:
    model = KMeans(n_clusters=k, random_state=42, n_init="auto").fit(X_scaled)
    inertia.append(model.inertia_)
    silhouette.append(silhouette_score(X_scaled, model.labels_))

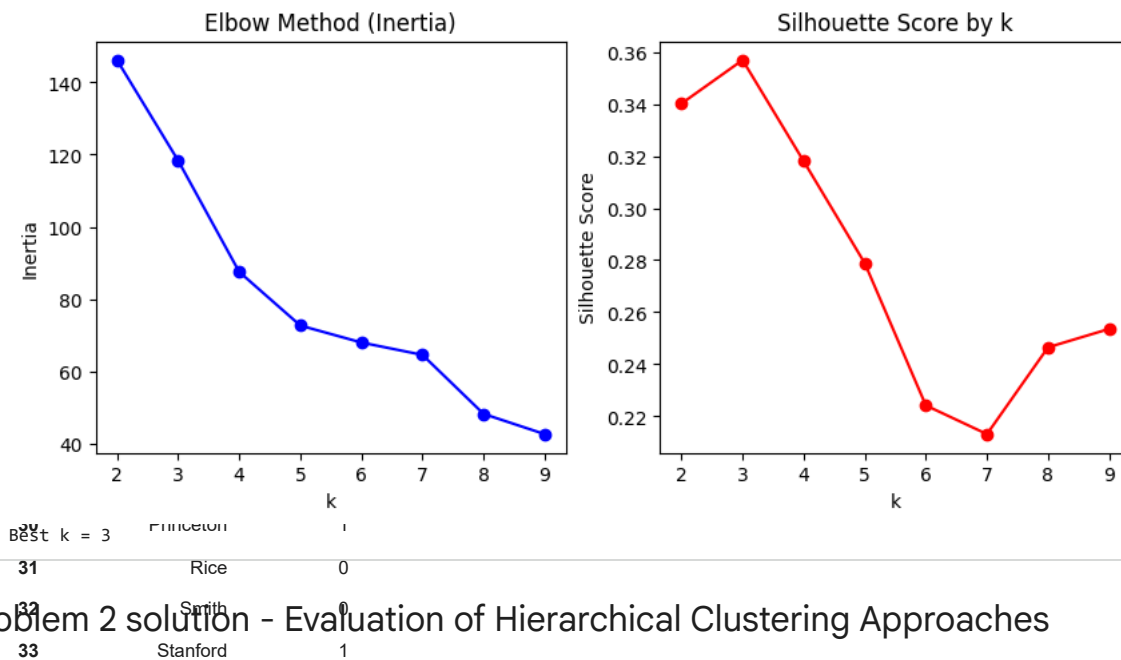
# plot elbow & silhouette
fig, ax1 = plt.subplots(1, 2, figsize=(10,4))
ax1[0].plot(K_range, inertia, 'bo-')
ax1[0].set_title("Elbow Method (Inertia)")
ax1[0].set_xlabel("k")
ax1[0].set_ylabel("Inertia")

ax1[1].plot(K_range, silhouette, 'ro-')
ax1[1].set_title("Silhouette Score by k")
ax1[1].set_xlabel("k")
ax1[1].set_ylabel("Silhouette Score")
plt.show()

# choose best k
best_k = K_range[np.argmax(silhouette)]
print(f"Best k = {best_k}")

# fit final model
kmeans = KMeans(n_clusters=best_k, random_state=42, n_init="auto")
kmeans.fit(X_scaled)
data["Cluster (KMeans)"] = kmeans.labels_

```



Problem 2 solution - Evaluation of Hierarchical Clustering Approaches

```

# quick copy to be safe
df = data.copy()

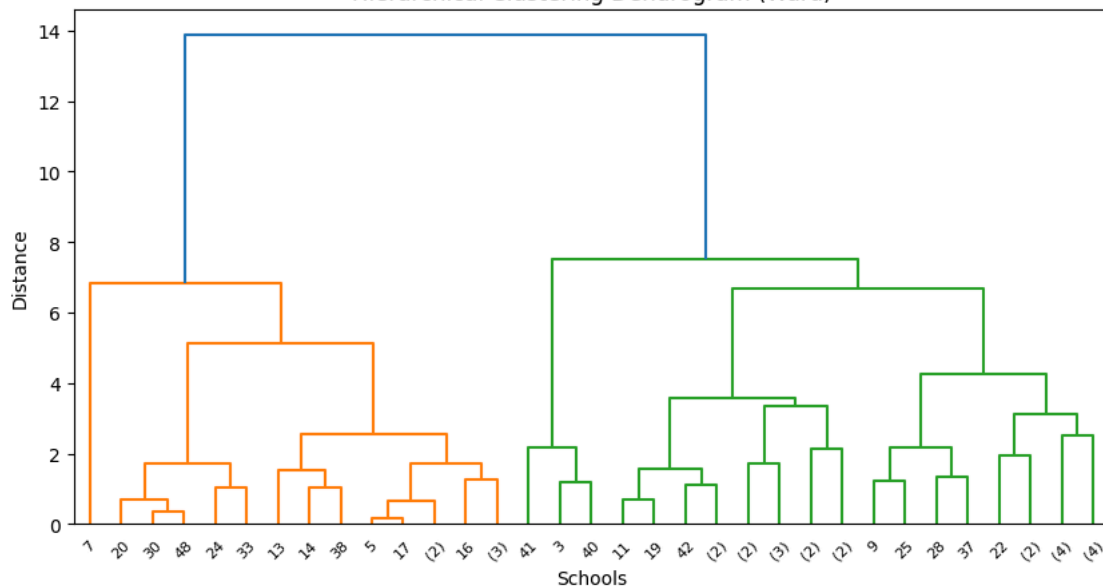
link_ward = linkage(X_scaled, method="ward")
plt.figure(figsize=(10, 5))
dendrogram(link_ward, truncate_mode="level", p=5)
plt.title("Hierarchical Clustering Dendrogram (Ward)")
plt.xlabel("Schools")
plt.ylabel("Distance")
plt.show()

agg_ward = AgglomerativeClustering(n_clusters=3, linkage="ward")
df["Cluster (Ward)"] = agg_ward.fit_predict(X_scaled)

```

43	Washington U (MO)	1
44	Washinton and Lee	0
45	Wellesley	0
46	Wesleyan (CT)	0
47	Williams	0
48	Yale	1

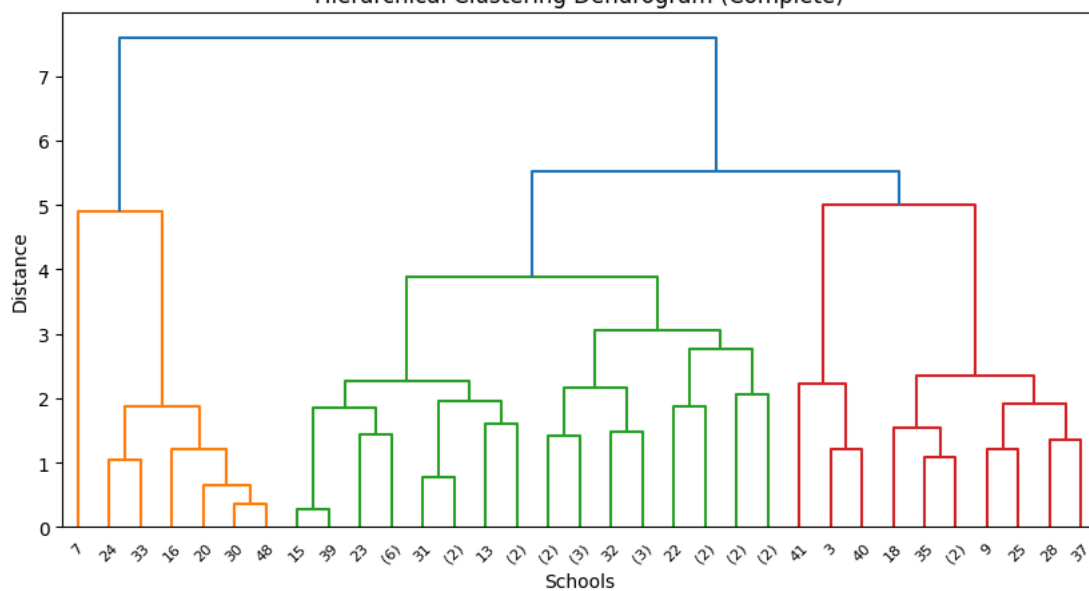
Hierarchical Clustering Dendrogram (Ward)



```
link_complete = linkage(X_scaled, method="complete")
plt.figure(figsize=(10, 5))
dendrogram(link_complete, truncate_mode="level", p=5)
plt.title("Hierarchical Clustering Dendrogram (Complete)")
plt.xlabel("Schools")
plt.ylabel("Distance")
plt.show()

agg_complete = AgglomerativeClustering(n_clusters=3, linkage="complete")
df["Cluster (Complete)"] = agg_complete.fit_predict(X_scaled)
```

Hierarchical Clustering Dendrogram (Complete)



```
# compare both methods
print("silhouette (Ward):", silhouette_score(X_scaled, df["Cluster (Ward)"]))
print("silhouette (Complete):", silhouette_score(X_scaled, df["Cluster (Complete)"]))

compare = df[["School", "Cluster (Ward)", "Cluster (Complete)"]]
print(compare.head(10))
```

```
silhouette (Ward): 0.3483433958190894
silhouette (Complete): 0.2632389477547466
```

	School	Cluster (Ward)	Cluster (Complete)
0	Amherst	0	2
1	Barnard	1	0
2	Bates	1	2
3	Berkeley	2	0
4	Bowdoin	0	2
5	Brown	0	2

6	Bryn Mawr	1	2
7	Cal Tech	0	1
8	Carleton	1	2
9	Carnegie Mellon	1	0

Compare and contrast the results of the two clustering methods.

Ward made tighter, clearer clusters (Silhouette = 0.35) than Complete (0.26). Ward grouped top schools like Amherst, Brown, and Cal Tech together. High SAT, high spending, high grad rates. Complete mixed more schools and was less consistent. Overall, Ward worked better for this dataset.

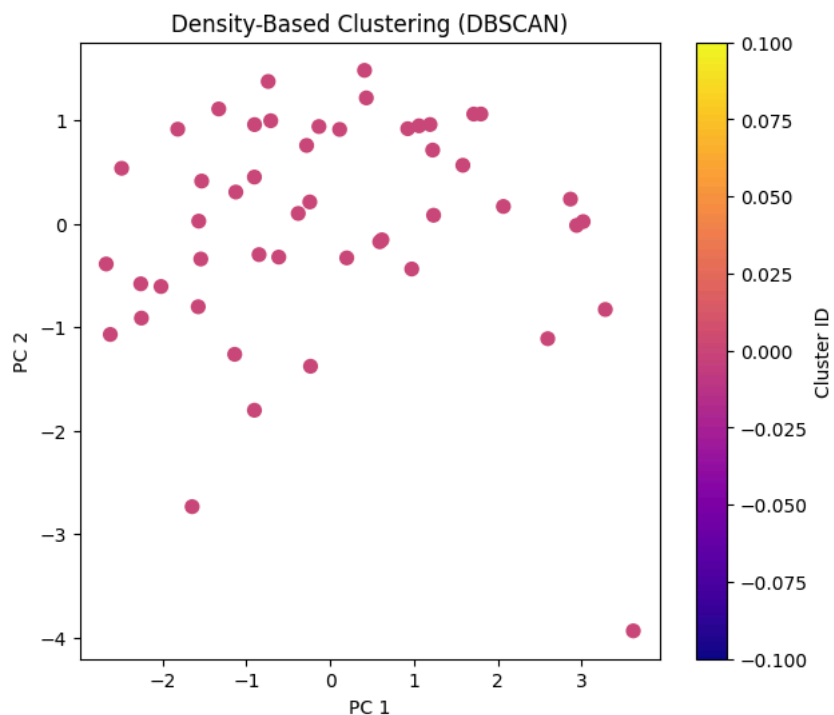
Problem 3 solution - Density-Based Clustering Analysis (DBSCAN)

```
dbscan = DBSCAN(eps=30, min_samples=6)
labels = dbscan.fit_predict(X_scaled)
df["Cluster (DBSCAN)"] = labels
```

```
# visual for HDBscan

pca = PCA(n_components=2)
X_2d = pca.fit_transform(X_scaled)

plt.figure(figsize=(7,6))
scatter = plt.scatter(X_2d[:,0], X_2d[:,1], c=labels, cmap="plasma", s=50)
plt.title("Density-Based Clustering (DBSCAN)")
plt.xlabel("PC 1")
plt.ylabel("PC 2")
plt.colorbar(scatter, label="Cluster ID")
plt.show()
```



```
# check cluster count

n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print(f"Clusters found: {n_clusters}")

if n_clusters > 1:
    sil = silhouette_score(X_scaled, labels)
    print(f"Silhouette Score (DBSCAN): {sil:.3f}")
else:
    print("Too few clusters for silhouette score - adjust eps/min_samples.")
```

```
Clusters found: 1
Too few clusters for silhouette score - adjust eps/min_samples.
```

```
#experimenting w diff eps parameters
for e in [15, 20, 25, 30, 35]:
    model = DBSCAN(eps=e, min_samples=6).fit(X_scaled)
    print(e, len(set(model.labels_)) - (1 if -1 in model.labels_ else 0))
```

```
15 1
20 1
25 1
30 1
35 1
```

```
np.random.seed(42)

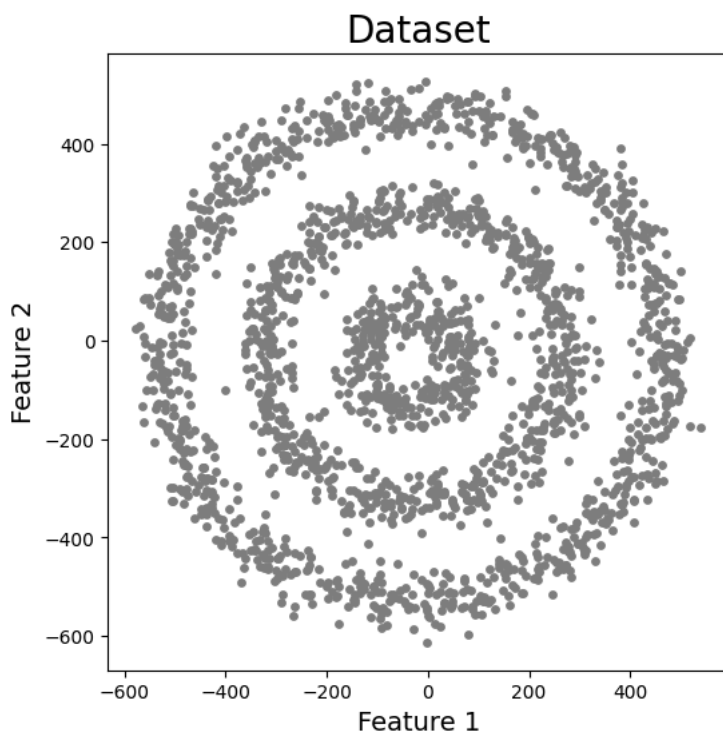
# Function for creating datapoints in the form of a circle
def PointsInCircum(r,n=100):
    return [(math.cos(2*math.pi/n*x)*r+np.random.normal(-30,30),math.sin(2*math.pi/n*x)*r+np.random.normal(-30,30)) for x in range(1,n+1)]

# Creating data points in the form of a circle
df1 = pd.DataFrame(PointsInCircum(500,1000))
df2 = pd.DataFrame(PointsInCircum(300,700))
df3 = pd.DataFrame(PointsInCircum(100,300))

# Adding noise to the dataset
df4 = pd.DataFrame([(np.random.randint(-600,600),np.random.randint(-600,600)) for i in range(300)])

#df = pd.concat([df1, df2, df3, df4], axis= 0)
df = pd.concat([df1, df2, df3], axis= 0)

plt.figure(figsize=(6,6))
plt.scatter(df[0],df[1],s=15,color='grey')
plt.title('Dataset',fontsize=20)
plt.xlabel('Feature 1',fontsize=14)
plt.ylabel('Feature 2',fontsize=14)
plt.show()
```



- Using the data in the DataFrame df above, conduct spectral clustering and provide a visualization of the clustering results. Be sure to scale your data first.

```
# Scale data
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)

n_clusters = 3
spectral = SpectralClustering(n_clusters=n_clusters, random_state=1, affinity='rbf', gamma=10) # Adjust gamma as needed
spectral_labels = spectral.fit_predict(df_scaled)
```